MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Faculty Name	Abhimanyu Saraswat		
Subject Name with Code	7CS5-13 Generative AI		
Semester	7th	**Session**	2025-26 (Odd)
Branch	CSE	**Batch**	A,B,AIDS,AIML
Unit No.	3	**Submission Date**	1-Oct-25

Lecture No.	*Topic Covered*	*Page No.*	*Total Pages*
20	CNN architecture	1 - 2	2
21	convolutional layer	3 - 11	9
22	CNN building blocks	12 - 16	5
23	Variants of the convolutional model	17 - 19	3
24	introduction to recurrent neural networks	20 - 22	3
25	Training through RNN	23 - 24	2

References:	
1	Generative Deep Learning: Teaching Machines to Paint, Write, Compose, and Play by David Foster, (
2	Deep Learning by Ian Goodfellow, Yoshua Bengio, and Aaron Courville
3	Generative Adversarial Networks by Ian Goodfellow, Yoshua Bengio, and Aaron Courville

Faculty

HOD 1/10/25

Dean Academics 4/10/25

Unit-3 Generative Models for Computer Vision

Introduction to Convolution Neural Networks for Image P.
Lecture1

CNN architecture

The Convolutional layer applies filters to the input image to extract features, the Pooling layer downsamples the image to reduce computation, and the fully connected layer makes the final prediction. The network learns the optimal filters through backpropagation and gradient descent.

Convolutional Neural Networks

A Convolutional Neural Network, also known as CNN or ConvNet, is a class of neural networks that specializes in processing data that has a grid-like topology, such as an image. A digital image is a binary representation of visual data. It contains a series of pixels arranged in a grid-like fashion that contains pixel values to denote how bright and what color each pixel should be.

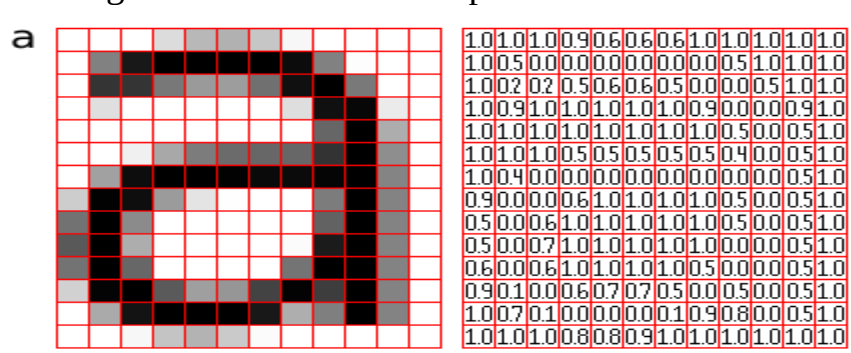


Figure 1: Representation of image as a grid of pixels)

The human brain processes a huge amount of information the second we see an image. Each neuron works in its own receptive field and is connected to other neurons in a way that they cover the entire visual field. Just as each neuron responds to stimuli only in the restricted region of the visual field called the receptive field in the biological vision system, each neuron in a CNN processes data only in its receptive field as well. The layers are arranged in such a way so that they detect

simpler patterns first (lines, curves, etc.) and more complex patterns (faces, objects, etc.) further along. By using a CNN, one can enable sight to computers.

Convolutional Neural Network Architecture

A CNN typically has three layers: a convolutional layer, a pooling layer, and a fully connected layer.

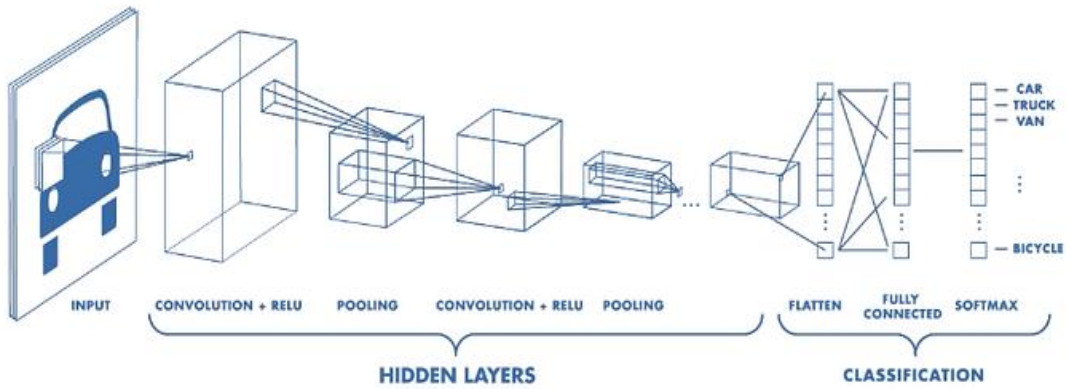


Figure 2: Architecture of a CNN

References

1. Generative Deep Learning: Teaching Machines to Paint, Write, Compose, and Play by David Foster, O'Reilly Media.
2. Deep Learning by Ian Goodfellow, Yoshua Bengio, and Aaron Courville.
3. Generative Adversarial Networks by Ian Goodfellow, Yoshua Bengio, and Aaron Courville.

Lecture2

convolutional layer

Convolution Layer

The convolution layer is the core building block of the CNN. It carries the main portion of the network's computational load.

This layer performs a dot product between two matrices, where one matrix is the set of learnable parameters otherwise known as a kernel, and the other matrix is the restricted portion of the receptive field. The kernel is spatially smaller than an image but is more in-depth. This means that, if the image is composed of three (RGB) channels, the kernel height and width will be spatially small, but the depth extends up to all three channels.

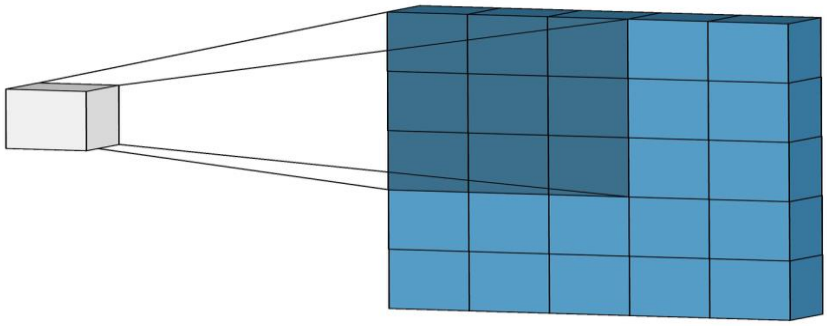


Illustration of Convolution Operation

During the forward pass, the kernel slides across the height and width of the image-producing the image representation of that receptive region. This produces a two-dimensional representation of the image known as an activation map that gives the response of the kernel at each spatial position of the image. The sliding size of the kernel is called a stride.

If we have an input of size W x W x D and Dout number of kernels with a spatial size of F with stride S and amount of padding P, then the size of output volume can be determined by the following formula:

$$W_{out} = \frac{W - F + 2P}{S} + 1$$

Formula for Convolution Layer

This will yield an output volume of size *Wout* x *Wou*t x *Dout*.

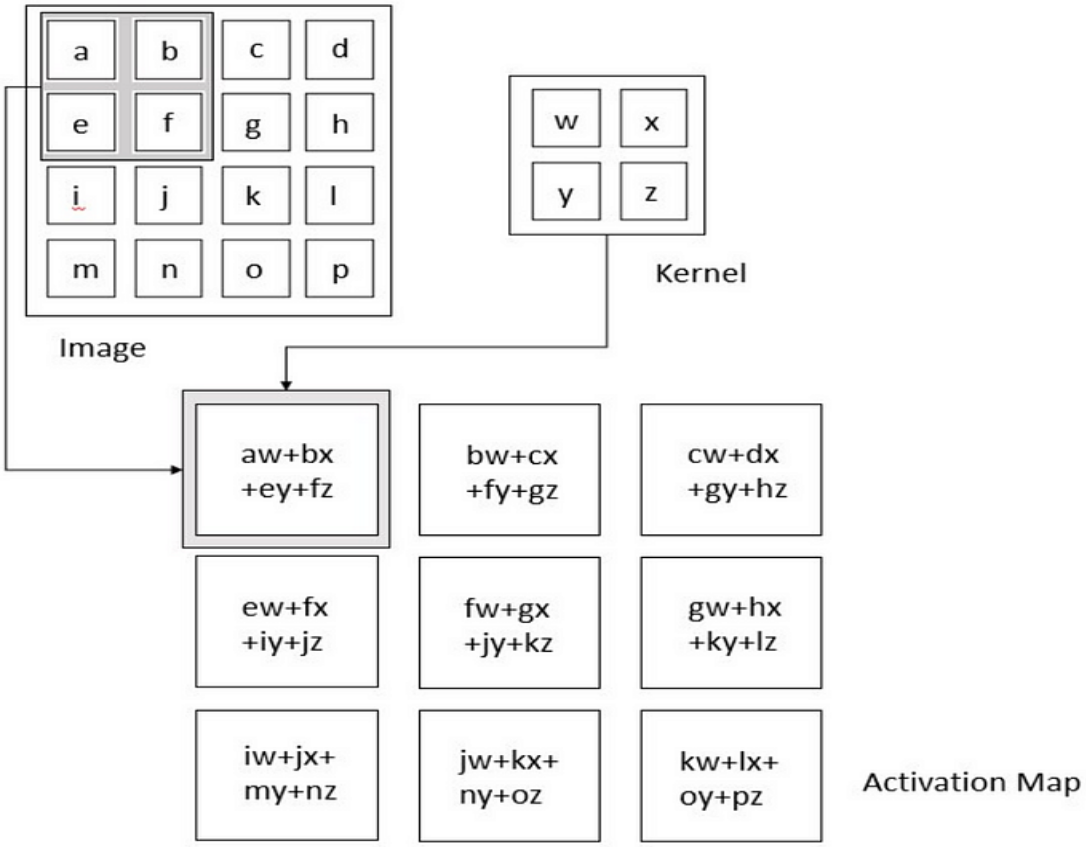


Figure 3: Convolution Operation

Motivation behind Convolution

Convolution leverages three important ideas that motivated computer vision researchers: sparse interaction, parameter sharing, and equivariant representation. Let's describe each one of them in detail.

Trivial neural network layers use matrix multiplication by a matrix of parameters describing the interaction between the input and output unit. This means that every output unit interacts with every input unit. However, convolution neural networks have *sparse interaction.* This is achieved by making kernel smaller than the input e.g., an image can have millions or thousands of pixels, but while processing it using kernel we can detect meaningful information that is of tens or hundreds of pixels. This means that we need to store fewer parameters that not only reduces the memory requirement of the model but also improves the statistical efficiency of the model.

If computing one feature at a spatial point (x1, y1) is useful then it should also be useful at some other spatial point say (x2, y2). It means that for a single two-dimensional slice i.e., for creating one activation map, neurons are constrained to use the same set of weights. In a traditional neural network, each element of the weight matrix is used once and then never revisited, while convolution network has *shared parameters* i.e., for getting output, weights applied to one input are the same as the weight applied elsewhere.

Due to parameter sharing, the layers of convolution neural network will have a property of *equivariance to translation*. It says that if we changed the input in a way, the output will also get changed in the same way.

Pooling Layer

The pooling layer replaces the output of the network at certain locations by deriving a summary statistic of the nearby outputs. This helps in reducing the spatial size of the representation, which decreases the required amount of computation and weights. The pooling operation is processed on every slice of the representation individually.

There are several pooling functions such as the average of the rectangular neighborhood, L2 norm of the rectangular neighborhood, and a weighted average based on the distance from the central pixel. However, the most popular process is max pooling, which reports the maximum output from the neighborhood.

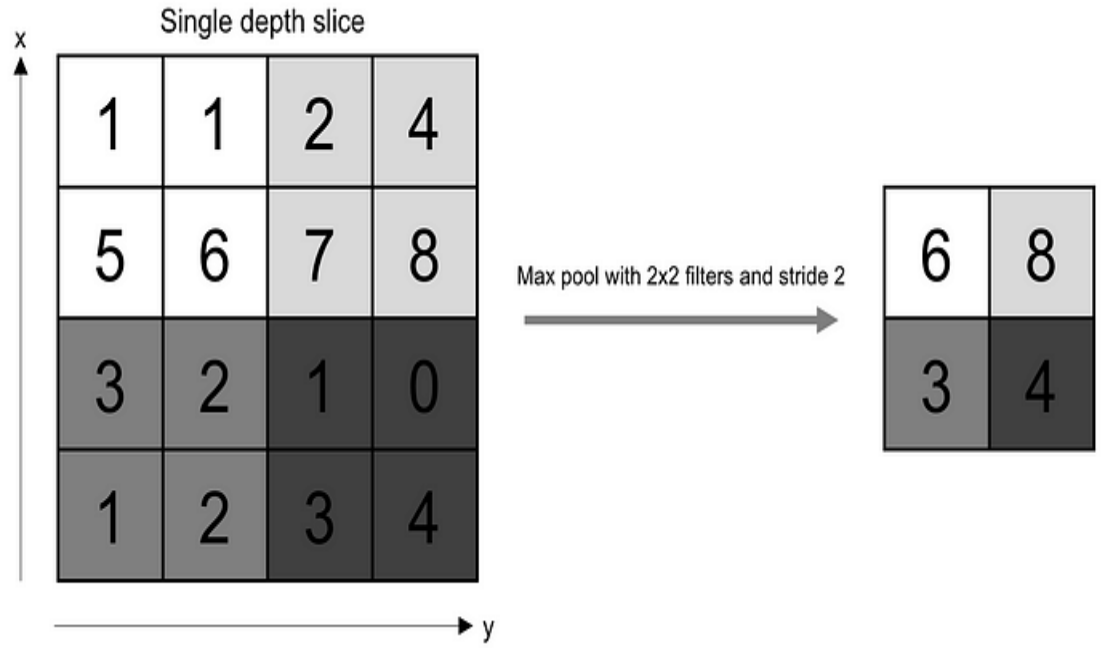


Figure 4: Pooling Operation

If we have an activation map of size *W* x *W* x *D*, a pooling kernel of spatial size *F*, and stride *S*, then the size of output volume can be determined by the following formula:

$$W_{out} = \frac{W - F}{S} + 1$$

Formula for Padding Layer

This will yield an output volume of size *Wout* x *Wout* x *D*.

In all cases, pooling provides some translation invariance which means that an object would be recognizable regardless of where it appears on the frame.

Fully Connected Layer

Neurons in this layer have full connectivity with all neurons in the preceding and succeeding layer as seen in regular FCNN. This is why it can be computed as usual by a matrix multiplication followed by a bias effect.

The FC layer helps to map the representation between the input and the output.

Non-Linearity Layers

Since convolution is a linear operation and images are far from linear, non-linearity layers are often placed directly after the convolutional layer to introduce non-linearity to the activation map.

There are several types of non-linear operations, the popular ones being:

1. Sigmoid

The sigmoid non-linearity has the mathematical form $\sigma(\kappa) = 1/(1+e^{-}\kappa)$. It takes a real-valued number and "squashes" it into a range between 0 and 1.

However, a very undesirable property of sigmoid is that when the activation is at either tail, the gradient becomes almost zero. If the local gradient becomes very small, then in backpropagation it will effectively "kill" the gradient. Also, if the data coming into the neuron is always positive, then the output of sigmoid will be either all positives or all negatives, resulting in a zig-zag dynamic of gradient updates for weight.

2. Tanh

Tanh squashes a real-valued number to the range [-1, 1]. Like sigmoid, the activation saturates, but — unlike the sigmoid neurons — its output is zero centered.

3. ReLU

The Rectified Linear Unit (ReLU) has become very popular in the last few years. It computes the function $f(\kappa)=\max(0,\kappa)$. In other words, the activation is simply threshold at zero.

In comparison to sigmoid and tanh, ReLU is more reliable and accelerates the convergence by six times.

Unfortunately, a con is that ReLU can be fragile during training. A large gradient flowing through it can update it in such a way that the neuron will never get further updated. However, we can work with this by setting a proper learning rate.

Designing a Convolutional Neural Network

Now that we understand the various components, we can build a convolutional neural network. We will be using Fashion-MNIST, which is a dataset of Zalando's article images consisting of a training set of 60,000 examples and a test set of 10,000 examples. Each example is a 28x28 grayscale image, associated with a label from 10 classes. The dataset can be downloaded here.

Our convolutional neural network has architecture as follows:

[INPUT]

→[CONV 1] → [BATCH NORM] → [ReLU] → [POOL 1]

→ [CONV 2] → [BATCH NORM] → [ReLU] → [POOL 2]

→ [FC LAYER] → [RESULT]

For both conv layers, we will use kernel of spatial size 5 x 5 with stride size 1 and padding of 2. For both pooling layers, we will use max pool operation with kernel size 2, stride 2, and zero padding.

CONV 1

Input Size ($W_1 \times H_1 \times D_1$) = 28 x 28 x 1

- Requires four hyperparameter:
  - Number of kernels, k = 16
  - Spatial extend of each one, F = 5
  - Stride Size, S = 1
  - Amount of zero padding, P = 2
- Outputting volume of $W_2 \times H_2 \times D_2$
  - $W_2 = (28 - 5 + 2 (2)) / 1 + 1 = 28$
  - $H_2 = (28 - 5 + 2 (2)) / 1 + 1 = 28$
  - $D_2 = k$

Output of Conv 1 ($W_2 \times H_2 \times D_2$) = 28 x 28 x 16

Calculations for Conv 1 Layer ()

POOL 1

Input Size ($W_2 \times H_2 \times D_2$) = 28 x 28 x 16

- Requires two hyperparameter:
  - Spatial extend of each one, F = 2
  - Stride Size, S = 2
- Outputting volume of $W_3 \times H_3 \times D_2$
  - $W_3 = (28 - 2) / 2 + 1 = 14$
  - $H_3 = (28 - 2) / 2 + 1 = 14$

Output of Pool 1 ($W_3 \times H_3 \times D_2$) = 14 x 14 x 16

Calculations for Pool1 Layer (Image by Author)

CONV 2
Input Size (W_3 x H_3 x D_2) = 14 x 14 x 16
• Requires four hyperparameter: o Number of kernels, k = 32 o Spatial extend of each one, F = 5 o Stride Size, S = 1 o Amount of zero padding, P = 2
• Outputting volume of W_4 x H_4 x D_3 o W_4 = (14 – 5 + 2 (2)) / 1 + 1 = 14 o H_4 = (14 – 5 + 2 (2)) / 1 + 1 = 14 o D_3 = k
Output of Conv 2 (W_4 x H_4 x D_3) = 14 x 14 x 32

Calculations for Conv 2 Layer (

Calculations for Pool2 Layer (Image by Author)

FC Layer
Input Size (W_5 x H_5 x D_3) = 7 x 7 x 32
Output Size (Number of Classes) = 10

We have also used batch normalization in our network, which saves us from improper initialization of weight matrices by explicitly forcing the network to take on unit Gaussian distribution. The code for the above-defined network is available here. We have trained using cross-entropy as our loss function and the Adam Optimizer with a learning rate of 0.001. After training the model, we achieved 90% accuracy on the test dataset.

Applications

Below are some applications of Convolutional Neural Networks used today:

1. Object detection: With CNN, we now have sophisticated models like R-CNN, Fast R-CNN, and Faster R-CNN that are the predominant pipeline for many object detection models deployed in autonomous vehicles, facial detection, and more.

2. Semantic segmentation: In 2015, a group of researchers from Hong Kong developed a CNN-based Deep Parsing Network to incorporate rich information into an image segmentation model. Researchers from UC Berkeley also built fully convolutional networks that improved upon state-of-the-art semantic segmentation.

3. Image captioning: CNNs are used with recurrent neural networks to write captions for images and videos. This can be used for many applications such as activity recognition or describing videos and images for the visually impaired. It has been heavily deployed by YouTube to make sense to the huge number of videos uploaded to the platform on a regular basis.

References

1. Generative Deep Learning: Teaching Machines to Paint, Write, Compose, and Play by David Foster, O'Reilly Media.
2. Deep Learning by Ian Goodfellow, Yoshua Bengio, and Aaron Courville.
3. Generative Adversarial Networks by Ian Goodfellow, Yoshua Bengio, and Aaron Courville.

Lecture3

CNN Building Blocks

Neural networks accept an input image/feature vector (one input node for each entry) and transform it through a series of hidden layers, commonly using nonlinear activation functions. Each hidden layer is also made up of a set of neurons, where each neuron is *fully connected* to all neurons in the previous layer. The last layer of a neural network (i.e., the "output layer") is also fully connected and represents the final output classifications of the network.

A diverse dataset is essential for training a CNN and understanding its different layer types. By training the network on the dataset, we can observe how each layer extracts features and contributes to the final prediction.

Layer Types

There are many types of layers used to build Convolutional Neural Networks, but the ones you are most likely to encounter include:

- Convolutional (

  CONV

)
- Activation (

  ACT

  or

  RELU

  , where we use the same or the actual activation function)
- Pooling (

  POOL

)

- Fully connected (

  FC

)
- Batch normalization (

  BN

)
- Dropout (

  DO

)

Stacking a series of these layers in a specific manner yields a CNN. We often use simple text diagrams to describe a CNN:

INPUT => CONV => RELU => FC => SOFTMAX
.

Here, we define a simple CNN that accepts an input, applies a convolution layer, then an activation layer, then a fully connected layer, and, finally, a softmax classifier to obtain the output classification probabilities. The

SOFTMAX
activation layer is often omitted from the network diagram as it is assumed it directly follows the final
FC.

Convolutional Layers

The CONV layer is the core building block of a Convolutional Neural Network. The CONV layer parameters consist of a set of *K* learnable filters (i.e., "kernels"), where each filter has a width and a height, and are nearly always square. These filters are small (in terms of their spatial dimensions) but extend throughout the full depth of the volume.

For inputs to the CNN, the depth is the number of channels in the image (i.e., a depth of three when working with RGB images, one for each channel). For volumes deeper in the network, the depth will be the number of filters applied in the *previous* layer.

To make this concept more clear, let's consider the forward-pass of a CNN, where we convolve each of the *K* filters across the width and height of the input volume. More simply, we can think of each of our *K* kernels sliding across the input region, computing an element-wise multiplication, summing, and then storing the output value in a 2-dimensional *activation map*, such as in **Figure 1**.

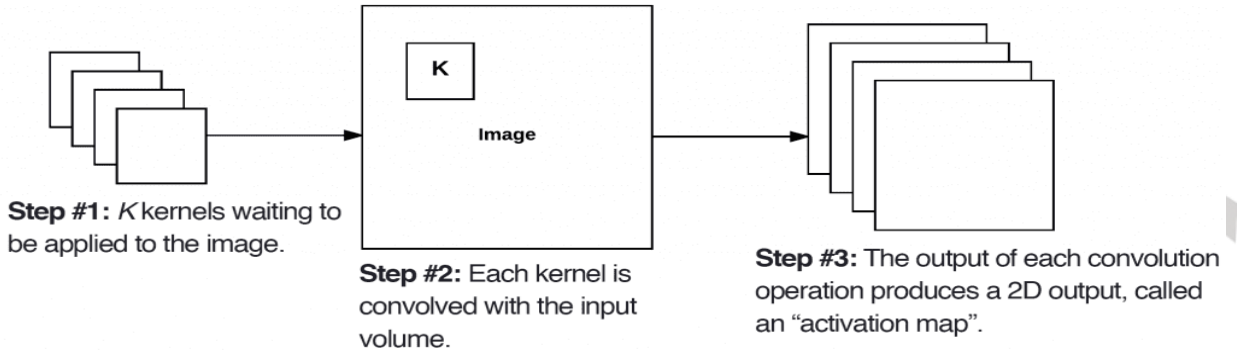


Figure 1: *Left:* At each convolutional layer in a CNN, there are *K* kernels applied to the input volume. *Middle:* Each of the *K* kernels is convolved with the input volume. *Right:* Each kernel produces a 2D output, called an *activation map*.

After applying all *K* filters to the input volume, we now have *K*, 2-dimensional activation maps. We then stack our *K* activation maps along the depth dimension of our array to form the final output volume **(Figure 2**).

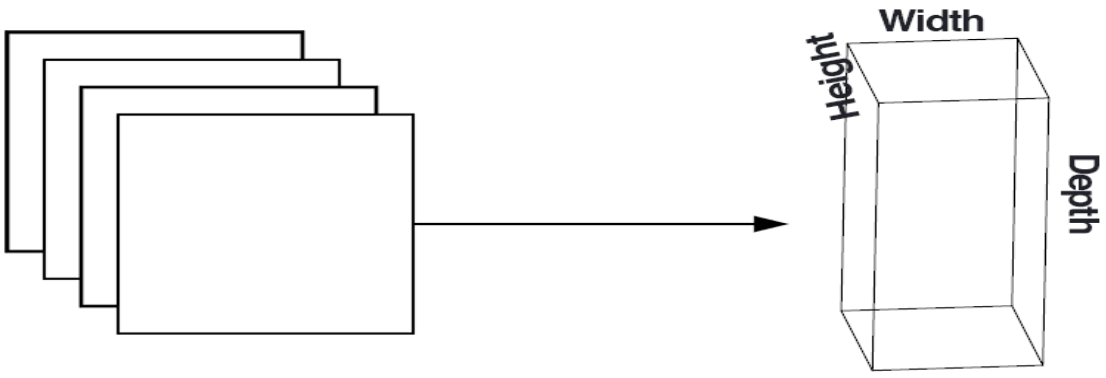


Figure 2: After obtaining the *K* activation maps, they are stacked together to form the input volume to the next layer in the network.

Every entry in the output volume is thus an output of a neuron that "looks" at only a small region of the input. In this manner, the network "learns" filters that activate when they see a specific type of feature at a *given spatial location* in the input volume. In lower layers of the network, filters may activate when they see edge-like or corner-like regions.

Then, in the deeper layers of the network, filters may activate in the presence of high-level features, such as parts of the face, the paw of a dog, the hood of a car, etc. This activation concept is as if these neurons are becoming "excited" and "activating" when they see a particular pattern in an input image.

The concept of convolving a small filter with a large(r) input volume has special meaning in Convolutional Neural Networks — specifically, the **local connectivity** and the **receptive field** of a neuron. When working with images, it's often impractical to connect neurons in the current volume to *all* neurons in the previous volume — there are simply too many connections and too many weights, making it impossible to train deep networks on images with large spatial dimensions. Instead, when utilizing CNNs, we choose to connect each neuron to only a *local region* of the input volume — we call the size of this local region the **receptive field** (or simply, the variable F) of the neuron.

To make this point clear, let's return to our CIFAR-10 dataset, where the input volume has an input size of $32 \times 32 \times 3$. Each image thus has a width of 32 pixels, a height of 32 pixels, and a depth of 3 (one for each RGB channel). If our receptive field is of size 3×3, then each neuron in the

CONV
layer will connect to a 3×3 local region of the image for a total of $3 \times 3 \times 3 = 27$ weights (remember, the depth of the filters is three because they extend through the full depth of the input image, in this case, three channels).

Now, let's assume that the spatial dimensions of our input volume have been reduced to a smaller size, but our depth is now larger, due to utilizing more filters deeper in the network, such that the volume size is now $16 \times 16 \times 94$. Again, if we assume a receptive field of size 3×3, then every neuron in the

CONV
layer will have a total of $3 \times 3 \times 94 = 846$ connections to the input volume. Simply put, the receptive field F is the ***size*** of the filter, yielding an $F \times F$ kernel that is convolved with the input volume.

At this point, we have explained the connectivity of neurons in the input volume, but not the arrangement or size of the output volume. There are three parameters that control the size of an output volume: the **depth**, **stride**, and **zero-padding** size, each of which we'll review below.

References

1. Generative Deep Learning: Teaching Machines to Paint, Write, Compose, and Play by David Foster, O'Reilly Media.
2. Deep Learning by Ian Goodfellow, Yoshua Bengio, and Aaron Courville.
3. Generative Adversarial Networks by Ian Goodfellow, Yoshua Bengio, and Aaron Courville.

Lecture4

Variants of the Convolution Model

Fully connected Layers

Neurons in

FC
layers are fully connected to all activations in the previous layer, as is the standard for feedforward neural networks.
FC
layers are *always* placed at the end of the network (i.e., we don't apply a
CONV
layer, then an
FC
layer, followed by another
CONV
) layer.

It's common to use one or two

FC
layers prior to applying the softmax classifier, as the following (simplified) architecture demonstrates:
→ Launch Jupyter Notebook on Google Colab

Batch Normalization

First introduced by **Ioffe and Szegedy in their 2015 paper, *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift***, batch normalization layers (or

BN
for short), as the name suggests, are used to normalize the activations of a given input volume before passing it into the next layer in the network.

If we consider x to be our mini-batch of activations, then we can compute the normalized $\hat{x}$ via the following equation:

$$\hat{x}_i = \frac{x_i - \mu_\beta}{\sqrt{\sigma_\beta^2 + \epsilon}} \quad \textbf{(4)}$$

During *training*, we compute the μ_β and σ_β over each mini-batch β, where:

$$\mu_\beta = \frac{1}{M}\sum_{i=1}^{m} x_i \qquad \sigma_\beta^2 = \frac{1}{M}\sum_{i=1}^{m}(x_i - \mu_\beta)^2 \quad \textbf{(5)}$$

Dropout

The last layer type we are going to discuss is dropout. Dropout is actually a form of *regularization* that aims to help prevent overfitting by increasing testing accuracy, perhaps at the expense of training accuracy. For each mini-batch in our training set, dropout layers, with probability *p*, randomly disconnect inputs from the preceding layer to the next layer in the network architecture.

Figure 6 visualizes this concept where we randomly disconnect with probability *p* = 0.5 the connections between two

FC

layers for a given mini-batch. Again, notice how half of the connections are severed for this mini-batch. After the forward and backward pass are computed for the mini-batch, we re-connect the dropped connections, and then sample another set of connections to drop.

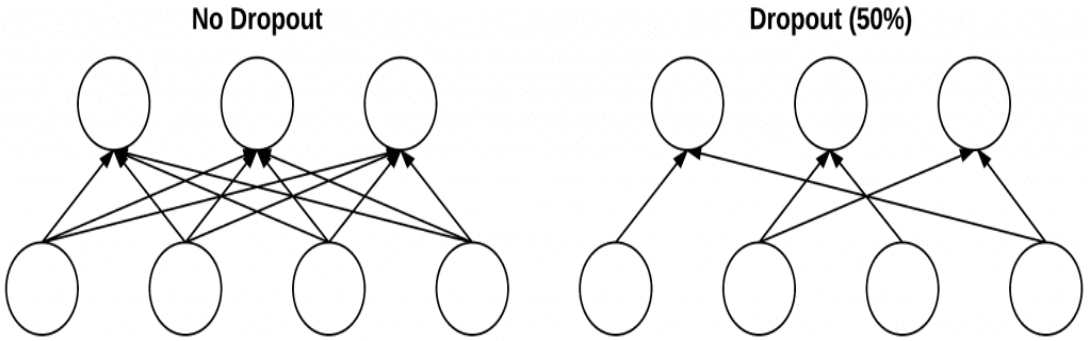


Figure 6: *Left:* Two layers of a neural network that are fully connected with no dropout. *Right:* The same two layers after dropping 50% of the connections.

The reason we apply dropout is to reduce overfitting by *explicitly* altering the network architecture at training time. Randomly dropping connections ensures that no single node in the network is responsible for "activating" when presented with a given pattern. Instead, dropout ensures there are *multiple, redundant nodes* that will activate when presented with similar inputs — this, in turn, helps our model to *generalize*.

References

1. Generative Deep Learning: Teaching Machines to Paint, Write, Compose, and Play by David Foster, O'Reilly Media.
2. Deep Learning by Ian Goodfellow, Yoshua Bengio, and Aaron Courville.
3. Generative Adversarial Networks by Ian Goodfellow, Yoshua Bengio, and Aaron Courville.

Lecture5
Introduction to Recurrent Neural Network

In this article, we will introduce a new variation of neural network which is the **Recurrent Neural Network** also known as **(RNN)** that works better than a simple neural network when data is sequential like Time-Series data and text data.

What is Recurrent Neural Network (RNN)?

Recurrent Neural Network(RNN) is a type of Neural Network where the output from the previous step is fed as input to the current step. In traditional neural networks, all the inputs and outputs are independent of each other, but in cases when it is required to predict the next word of a sentence, the previous words are required and hence there is a need to remember the previous words. Thus RNN came into existence, which solved this issue with the help of a Hidden Layer. The main and most important feature of RNN is its **Hidden state**, which remembers some information about a sequence. The state is also referred to as *Memory State* since it remembers the previous input to the network. It uses the same parameters for each input as it performs the same task on all the inputs or hidden layers to produce the output. This reduces the complexity of parameters, unlike other neural networks.

Architecture Of Recurrent Neural Network

RNNs have the same input and output architecture as any other deep neural architecture. However, differences arise in the way information flows from input to output. Unlike Deep neural networks where we have different weight matrices for each Dense network in RNN, the weight across the network remains the same. It calculates state hidden state H_i for every input $\mathbf{X_i}$. **By using the following formulas:**

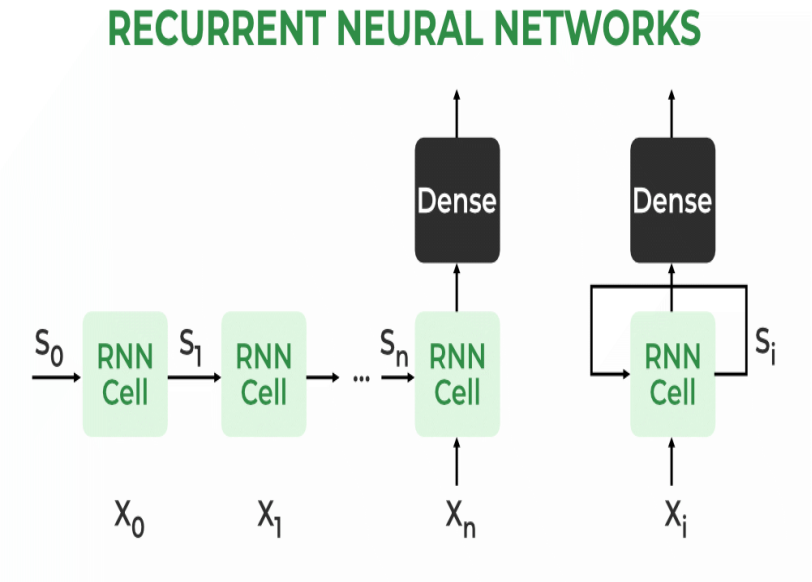
$$h = \sigma(UX + Wh_{-1} + B)$$

$$Y = O(Vh + C) \text{ Hence}$$

$$Y = f(X, h, W, U, V, B, C)$$

Here S is the State matrix which has element si as the state of the network at timestep i

The parameters in the network are W, U, V, c, b which are shared across timestep



What is Recurrent Neural Network

How RNN works

The Recurrent Neural Network consists of multiple fixed activation function units, one for each time step. Each unit has an internal state which is called the hidden state of the unit. This hidden state signifies the past knowledge that the network currently holds at a given time step. This hidden state is updated at every time step to signify the change in the knowledge of the network about the past. The hidden state is updated using the following recurrence relation:-

The formula for calculating the current state:

$$h_t = f(h_{t-1}, x_t)$$

where:

h_t -> current state
h_{t-1} -> previous state
x_t -> input state

Formula for applying Activation function(tanh):

$$h_t = \tanh (W_{hh}h_{t-1} + W_{xh}x_t)$$

where:

w_{hh} -> weight at recurrent neuron
w_{xh} -> weight at input neuron

The formula for calculating output:

$$y_t = W_{hy} h_t$$

Y_t -> output
W_{hy} -> weight at output layer
These parameters are updated using Backpropagation. However, since RNN works on sequential data here we use an updated backpropagation which is known as Backpropagation through time.

Backpropagation Through Time (BPTT)
In RNN the neural network is in an ordered fashion and since in the ordered network each variable is computed one at a time in a specified order like first h1 then h2 then h3 so on. Hence we will apply backpropagation throughout all these hidden time states sequentially.

Backpropagation Through Time (BPTT) In RNN

L(θ)(loss function) depends on h3

h3 in turn depends on h2 and W

h2 in turn depends on h1 and W

h1 in turn depends on h0 and W

where h0 is a constant starting state.

References

1. Generative Deep Learning: Teaching Machines to Paint, Write, Compose, and Play by David Foster, O'Reilly Media.
2. Deep Learning by Ian Goodfellow, Yoshua Bengio, and Aaron Courville.
3. Generative Adversarial Networks by Ian Goodfellow, Yoshua Bengio, and Aaron Courville.

Lecture6

Training through RNN

1. A single-time step of the input is provided to the network.
2. Then calculate its current state using a set of current input and the previous state.
3. The current ht becomes ht-1 for the next time step.
4. One can go as many time steps according to the problem and join the information from all the previous states.
5. Once all the time steps are completed the final current state is used to calculate the output.
6. The output is then compared to the actual output i.e the target output and the error is generated.
7. The error is then back-propagated to the network to update the weights and hence the network (RNN) is trained using Backpropagation through time.

Advantages of Recurrent Neural Network

1. An RNN remembers each and every piece of information through time. It is useful in time series prediction only because of the feature to remember previous inputs as well. This is called Long Short Term Memory.
2. Recurrent neural networks are even used with convolutional layers to extend the effective pixel neighborhood.

Disadvantages of Recurrent Neural Network

1. Gradient vanishing and exploding problems.
2. Training an RNN is a very difficult task.
3. It cannot process very long sequences if using tanh or relu as an activation function.

Applications of Recurrent Neural Network

1. Language Modelling and Generating Text
2. Speech Recognition
3. Machine Translation
4. Image Recognition, Face detection
5. Time series Forecasting

Types Of RNN

There are four types of RNNs based on the number of inputs and outputs in the network.

1. One to One
2. One to Many
3. Many to One
4. Many to Many

One to One

This type of RNN behaves the same as any simple Neural network it is also known as Vanilla Neural Network. In this Neural network, there is only one input and one output.

References

1. Generative Deep Learning: Teaching Machines to Paint, Write, Compose, and Play by David Foster, O'Reilly Media.
2. Deep Learning by Ian Goodfellow, Yoshua Bengio, and Aaron Courville.
3. Generative Adversarial Networks by Ian Goodfellow, Yoshua Bengio, and Aaron Courville.

Lecture7

The encoder

The encoder is an RNN that reads each symbol of an input sequence x sequentially. As it reads each symbol, the hidden state of the RNN changes according to Eq. (1). After reading the end of the sequence (marked by an end-of-sequence symbol), the hidden state of the RNN is a summary c of the whole input sequence

Encoder-Decoder Recurrent Neural Network Models for Neural Machine Translation

The encoder-decoder architecture for recurrent neural networks is the standard neural machine translation method that rivals and in some cases outperforms classical statistical machine translation methods.

This architecture is very new, having only been pioneered in 2014, although, has been adopted as the core technology inside Google's translate service.

In this post, you will discover the two seminal examples of the encoder-decoder model for neural machine translation.

After reading this post, you will know:

- The encoder-decoder recurrent neural network architecture is the core technology inside Google's translate service.
- The so-called "*Sutskever model*" for direct end-to-end machine translation.
- The so-called "*Cho model*" that extends the architecture with GRU units and an attention mechanism.

Kick-start your project with my new book Deep Learning for Natural Language Processing, including *step-by-step tutorials* and the *Python source code* files for all examples.

Encoder-Decoder Architecture for NMT

The Encoder-Decoder architecture with recurrent neural networks has become an effective and standard approach for both neural machine translation (NMT) and sequence-to-sequence (seq2seq) prediction in general.

The key benefits of the approach are the ability to train a single end-to-end model directly on source and target sentences and the ability to handle variable length input and output sequences of text.

As evidence of the success of the method, the architecture is the core of the Google translation service.

Our model follows the common sequence-to-sequence learning framework with attention. It has three components: an encoder network, a decoder network, and an attention network.

— Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation, 2016

In this post, we will take a closer look at two different research projects that developed the same Encoder-Decoder architecture at the same time in 2014 and achieved results that put the spotlight on the approach. They are:

- Sutskever NMT Model
- Cho NMT Model

References

1. Generative Deep Learning: Teaching Machines to Paint, Write, Compose, and Play by David Foster, O'Reilly Media.
2. Deep Learning by Ian Goodfellow, Yoshua Bengio, and Aaron Courville.
3. Generative Adversarial Networks by Ian Goodfellow, Yoshua Bengio, and Aaron Courville.

Lecture8

Sequence to Sequence (often abbreviated to seq2seq) models

Sequence to Sequence (often abbreviated to seq2seq) models is a special class of Recurrent Neural Network architectures that we typically use (but not restricted) to solve complex Language problems like Machine Translation, Question Answering, creating Chatbots, Text Summarization, etc.

A Simple Introduction to Sequence to Sequence Models

guest_blog — Updated On November 9th, 2020

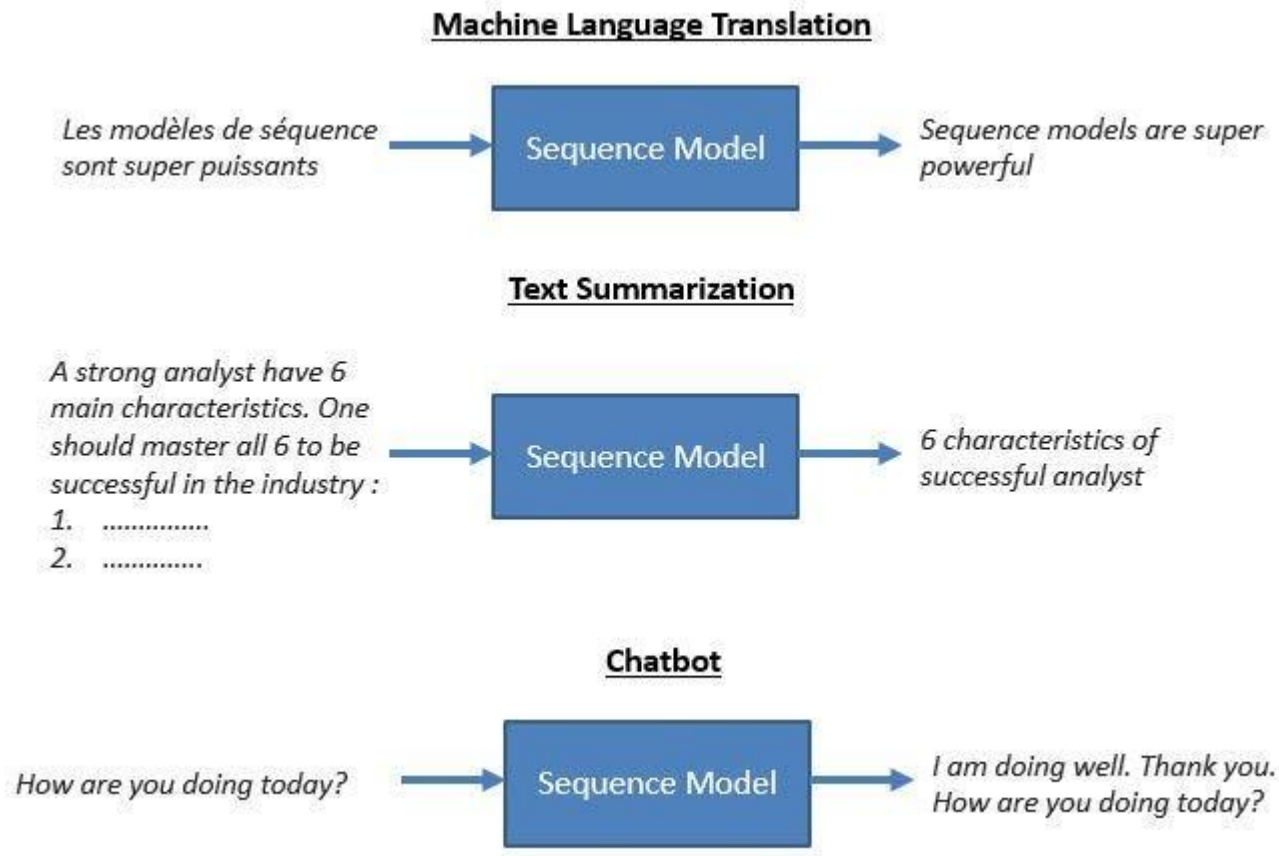
Algorithm Deep Learning Intermediate NLP Supervised Text Unstructured Data

Overview

In this article, I would give you an overview of sequence to sequence models which became quite popular for different tasks like machine translation, video captioning, image captioning, question answering, etc..

Introduction:

Sequence to Sequence (often abbreviated to seq2seq) models is a special class of Recurrent Neural Network architectures that we typically use (but not restricted) to solve complex Language problems like Machine Translation, Question Answering, creating Chatbots, Text Summarization, etc.

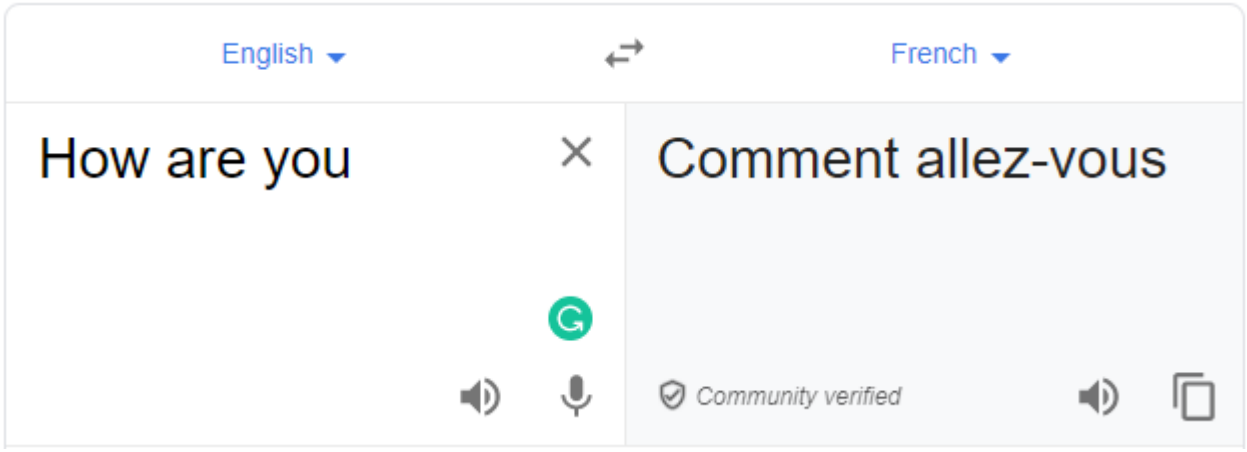


Source

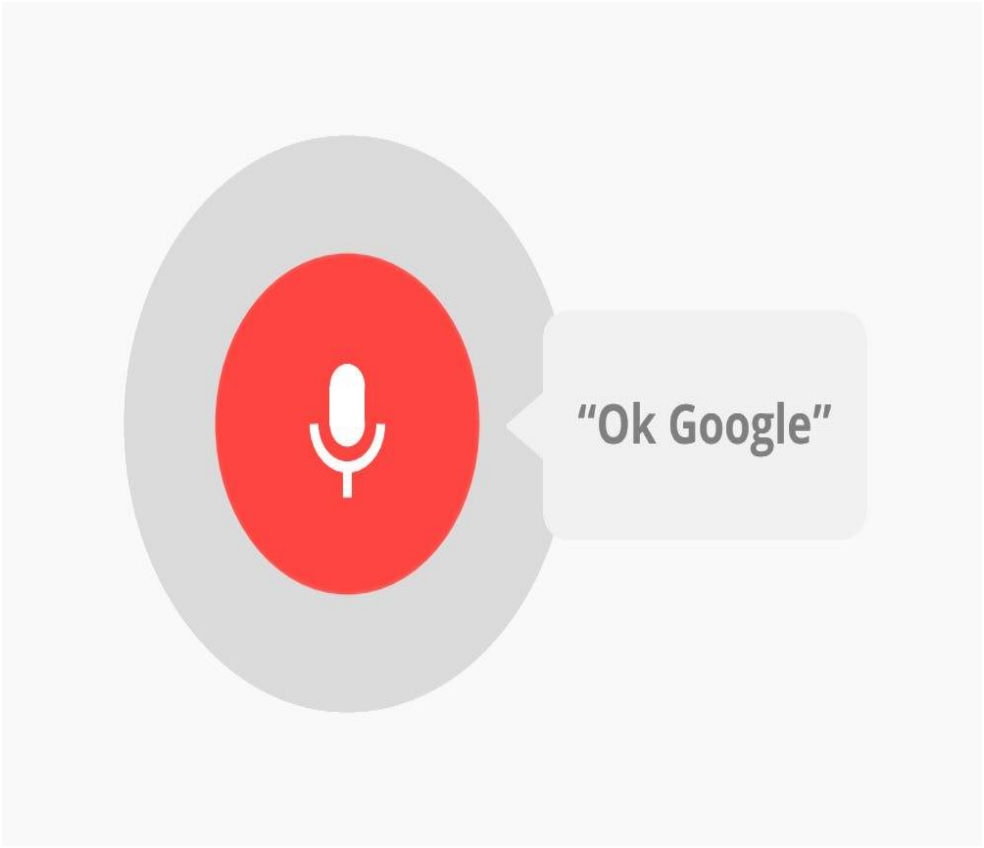
Use Cases of the Sequence to Sequence Models

Sequence to sequence models lies behind numerous systems that you face on a daily basis. For instance, seq2seq model powers applications like Google Translate, voice-enabled devices, and online chatbots. The following are some of the applications:

- *Machine translation* — a 2016 paper from Google shows how the seq2seq model's translation quality "approaches or surpasses all currently published results".



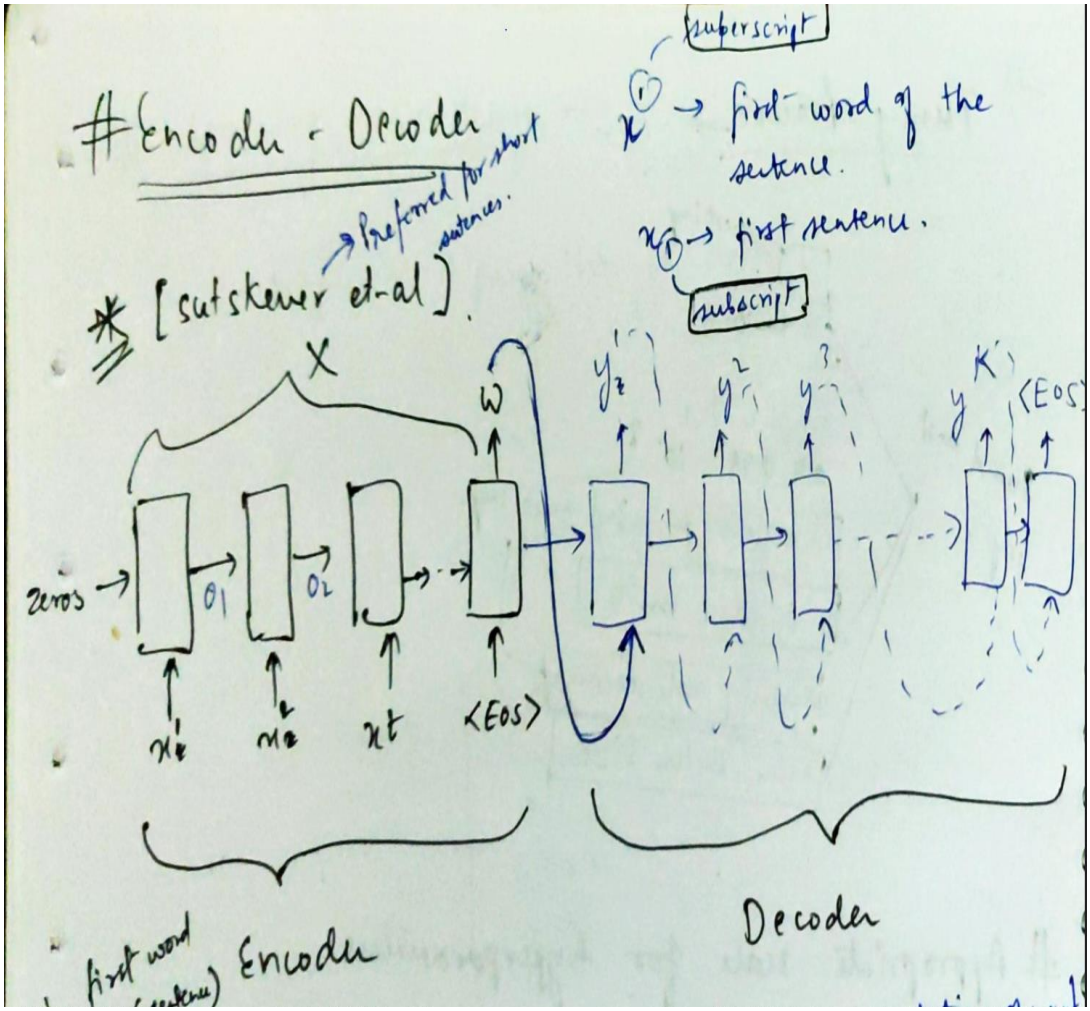
- *Speech recognition* — another Google paper that compares the existing seq2seq models on the speech recognition task.



These are only some applications where seq2seq is seen as the best solution. This model can be used as a solution to any sequence-based problem, especially ones where the inputs and outputs have different sizes and categories.

Encoder-Decoder Architecture:

The most common architecture used to build Seq2Seq models is Encoder-Decoder architecture.



Ilya Sutskever model for Sequence to Sequence Learning with Neural Networks

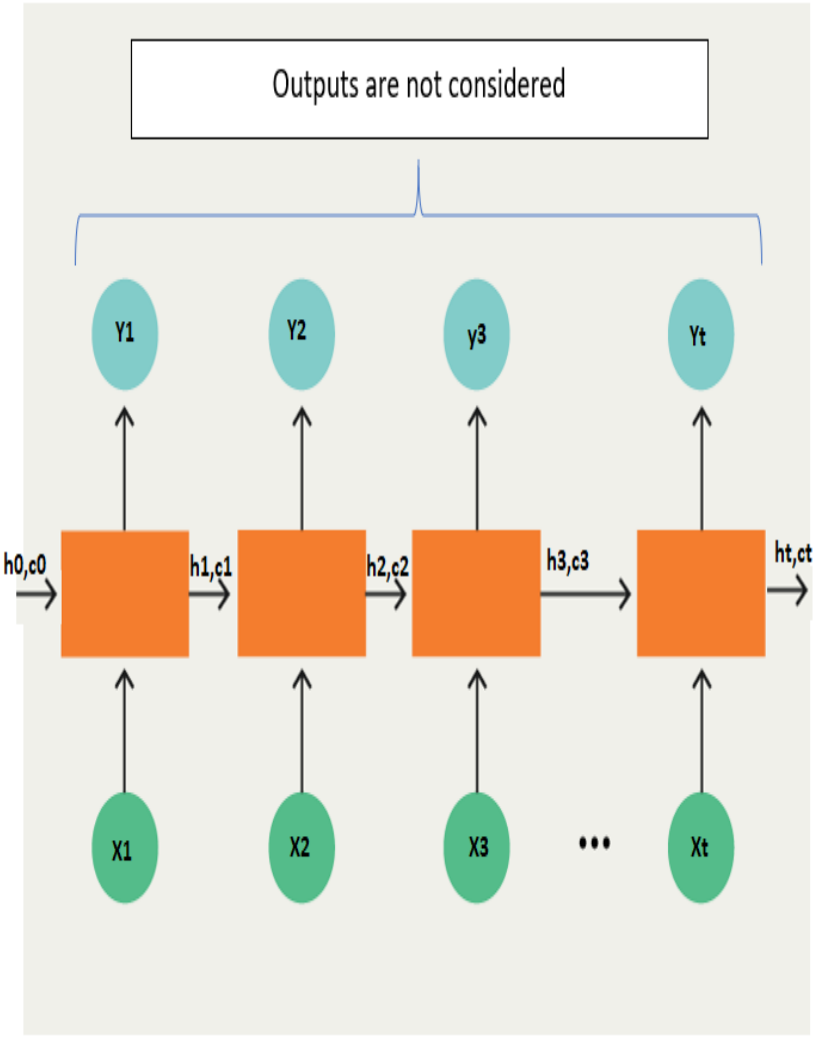
As the name implies, there are two components — an encoder and a decoder.

Encoder :

- Both encoder and the decoder are LSTM models (or sometimes GRU models)
- Encoder reads the input sequence and summarizes the information in something called the **internal state vectors** or **context vector** (in case of LSTM these are called the hidden state and cell state vectors). We discard the outputs of the encoder and only preserve the internal states. This context vector aims to encapsulate the information for all input elements in order to help the decoder make accurate predictions.

- The hidden states *h_i* are computed using the formula:

$$h_t = f(W^{(hh)} h_{t-1} + W^{(hx)} x_t)$$



The LSTM reads the data, one sequence after the other. Thus if the input is a sequence of length 't', we say that LSTM reads it in 't' time steps.

1. Xi = Input sequence at time step i.

2. hi and ci = LSTM maintains two states ('h' for hidden state and 'c' for cell state) at each time step. Combined together these are internal state of the LSTM at time step i.

3. Yi = Output sequence at time step i. Yi is actually a probability distribution over the entire vocabulary which is generated by using a softmax activation. Thus each Yi is a vector of size "vocab_size" representing a probability distribution.

Decoder :

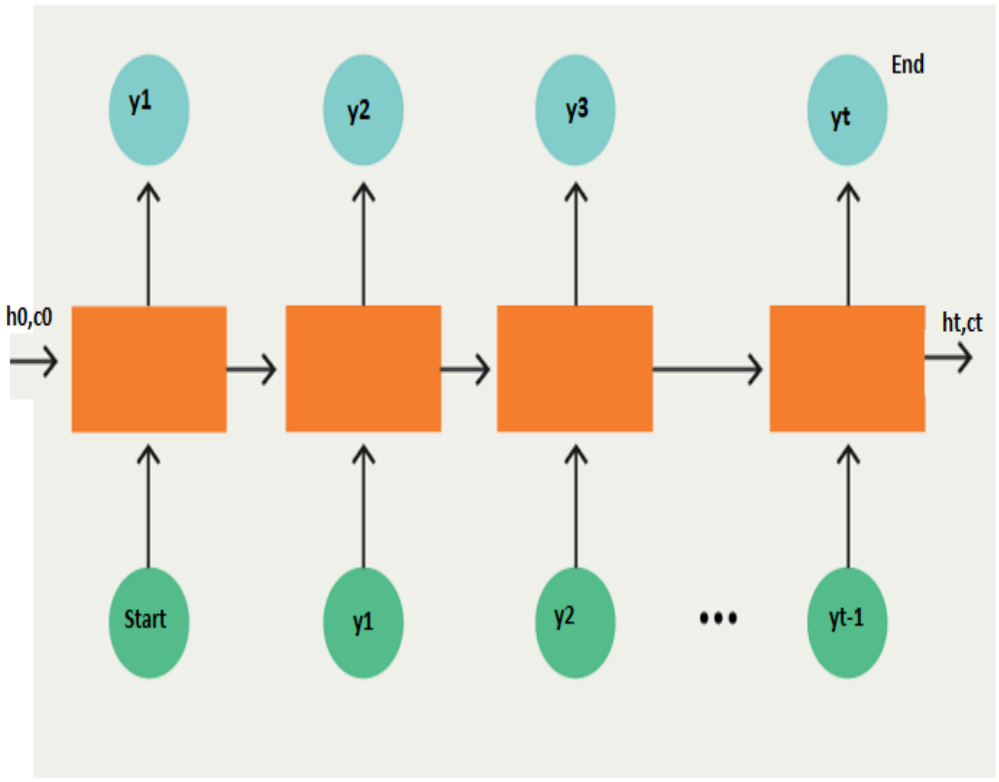
- The decoder is an LSTM whose initial states are initialized to the final states of the Encoder LSTM, i.e. the context vector of the encoder's final cell is input to the first cell of the decoder network. Using these initial states, the decoder starts generating the output sequence, and these outputs are also taken into consideration for future outputs.
- A stack of several LSTM units where each predicts an output y_t at a time step t.
- Each recurrent unit accepts a hidden state from the previous unit and produces and output as well as its own hidden state.
- Any hidden state *h_i* is computed using the formula:

$$h_t = f(W^{(hh)} h_{t-1})$$

- The output *y_t* at time step *t* is computed using the formula:

$$y_t = softmax(W^S h_t)$$

- We calculate the outputs using the hidden state at the current time step together with the respective weight W(S). Softmax is used to create a probability vector which will help us determine the final output (e.g. word in the question-answering problem).



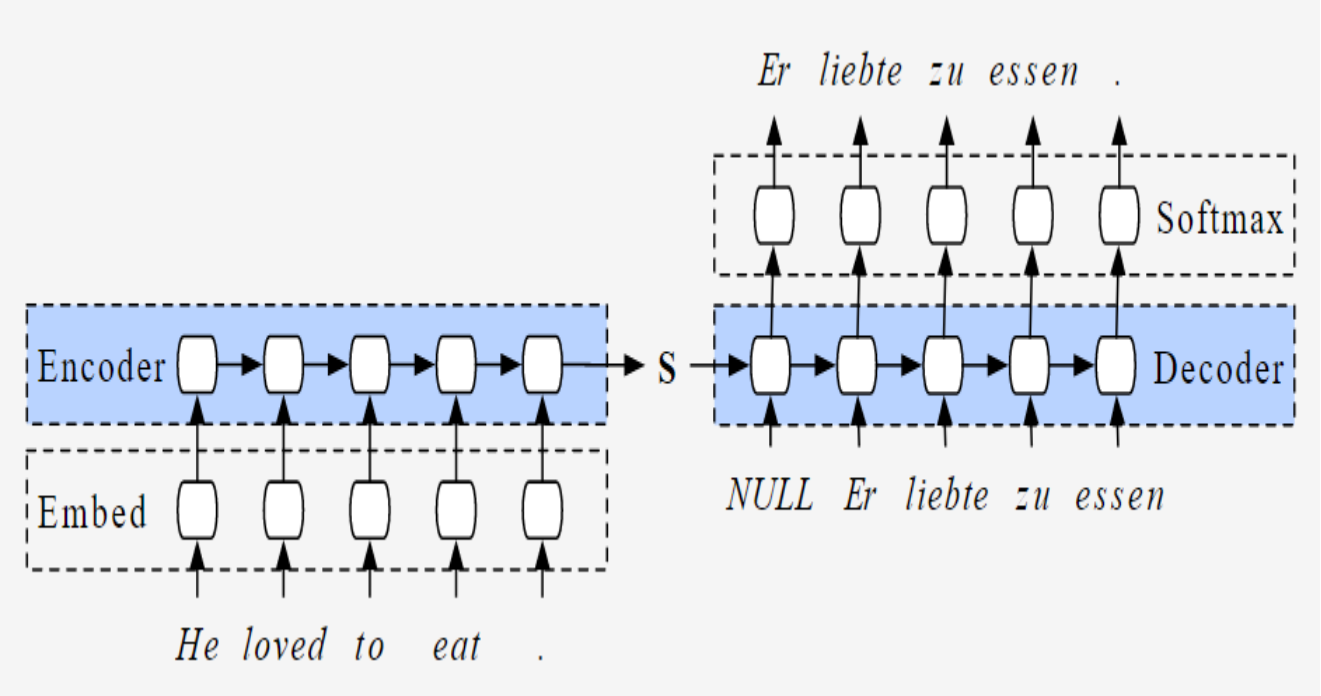
We will add two tokens in the output sequence as follows:

Example:

"**START_** John is hard working **_END**".

The most important point is that the initial states (h0, c0) of the decoder are set to the final states of the encoder. This intuitively means that the decoder is trained to start generating the output sequence depending on the information encoded by the encoder.

Finally, the loss is calculated on the predicted outputs from each time step and the errors are backpropagated through time in order to update the parameters of the network. Training the network over a longer period with a sufficiently large amount of data results in pretty good predictions.



Overall Encoder-Decoder Architecture

- During inference, we generate one word at a time.
- The initial states of the decoder are set to the final states of the encoder.
- The initial input to the decoder is always the START token.
- At each time step, we preserve the states of the decoder and set them as initial states for the next time step.
- At each time step, the predicted output is fed as input in the next time step.
- We break the loop when the decoder predicts the END token.

Drawbacks of Encoder-Decoder Models :

There are two primary drawbacks to this architecture, both related to length.

1. Firstly, as with humans, this architecture has very limited memory. That final hidden state of the LSTM, which we call **S or W**, is where you're trying to cram the entirety of the sentence you have to translate.**S or W** is usually only a few hundred units (read: floating-point numbers) long — the more you try to force into this fixed dimensionality vector, the lossier the neural network is

forced to be. Thinking of neural networks in terms of the "lossy compression" they're required to perform is sometimes quite useful.

2. Second, as a general rule of thumb, the deeper a neural network is, the harder it is to train. For recurrent neural networks, the longer the sequence is, the deeper the neural network is along the time dimension.This results in vanishing gradients, where the gradient signal from the objective that the recurrent neural network learns from disappears as it travels backward. Even with RNNs specifically made to help prevent vanishing gradients, such as the LSTM, this is still a fundamental problem.

Furthermore, for more robust and lengthy sentences we have models like **Attention Models** and **Transformers**.

Here ***this*** is my GitHub repository for complete word level as well as character level encoder-decoder Model.

References

1. Generative Deep Learning: Teaching Machines to Paint, Write, Compose, and Play by David Foster, O'Reilly Media.
2. Deep Learning by Ian Goodfellow, Yoshua Bengio, and Aaron Courville.
3. Generative Adversarial Networks by Ian Goodfellow, Yoshua Bengio, and Aaron Courville.

Lecture9

Deep recurrent neural networks

Deep recurrent neural networks are used as a means for nonlinear system identification. It is shown that state space models can be transformed into recurrent neural networks and vice versa.

Deep recurrent neural networks are used as a means for nonlinear system identification. It is shown that state space models can be transformed into recurrent neural networks and vice versa. This transformation and the understanding of the long short-term memory cell in terms of common system identification nomenclature makes the advances in deep learning more accessible to the controls and system identification communities. A systematic study of deep recurrent neural networks is carried out on a state-of-the-art system identification benchmark. The results indicate that if high amounts of data are available, standard recurrent neural networks achieve comparable performance to state-of-the-art system identification methods.

References

1. Generative Deep Learning: Teaching Machines to Paint, Write, Compose, and Play by David Foster, O'Reilly Media.
2. Deep Learning by Ian Goodfellow, Yoshua Bengio, and Aaron Courville.
3. Generative Adversarial Networks by Ian Goodfellow, Yoshua Bengio, and Aaron Courville.

Lecture10

Variation Of Recurrent Neural Network (RNN)

To overcome the problems like vanishing gradient and exploding gradient descent several new advanced versions of RNNs are formed some of these are as ;

1. Bidirectional Neural Network (BiNN)
2. Long Short-Term Memory (LSTM)

Bidirectional Neural Network (BiNN)

A BiNN is a variation of a Recurrent Neural Network in which the input information flows in both direction and then the output of both direction are combined to produce the input. BiNN is useful in situations when the context of the input is more important such as Nlp tasks and Time-series analysis problems.

Long Short-Term Memory (LSTM)

Long Short-Term Memory works on the read-write-and-forget principle where given the input information network reads and writes the most useful information from the data and it forgets about the information which is not important in predicting the output. For doing this three new gates are introduced in the RNN. In this way, only the selected information is passed through the network.

Difference between RNN and Simple Neural Network

RNN is considered to be the better version of deep neural when the data is sequential. There are significant differences between the RNN and deep neural networks they are listed as:

Recurrent Neural Network	Deep Neural Network
Weights are same across all the layers number of a Recurrent Neural Network	Weights are different for each layer of the network
Recurrent Neural Networks are used when the data is sequential and the number of inputs is not predefined.	A Simple Deep Neural network does not have any special method for sequential data also here the the number of inputs is fixed
The Numbers of parameter in the RNN are higher than in simple DNN	The Numbers of Parameter are lower than RNN

Recurrent Neural Network	Deep Neural Network
Exploding and vanishing gradients is the the major drawback of RNN	These problems also occur in DNN but these are not the major problem with DNN

References

1. Generative Deep Learning: Teaching Machines to Paint, Write, Compose, and Play by David Foster, O'Reilly Media.
2. Deep Learning by Ian Goodfellow, Yoshua Bengio, and Aaron Courville.
3. Generative Adversarial Networks by Ian Goodfellow, Yoshua Bengio, and Aaron Courville.